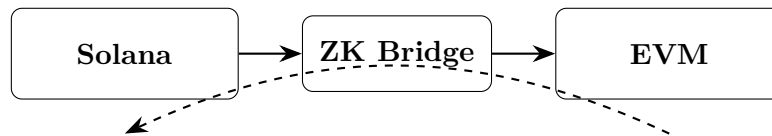


Meridian Link

Trust-Minimized Cross-Chain Token Transfers
Between Solana and EVM Chains via ZK Compression

Jayanth Kumar Morem

`jayanthkumar1903@gmail.com`



Version 1.0

January 2026

Technical Whitepaper

<https://github.com/RippnerLabs/meridian-link>

Abstract

Cross-chain bridges represent one of the most critical yet vulnerable components of blockchain infrastructure, with over \$2.5 billion lost to bridge exploits between 2022–2023 alone. MERIDIAN LINK introduces an architecture combining Light Protocol’s ZK Compression on Solana with GROTH16 zero-knowledge proofs for verification on EVM chains, reducing trust assumptions compared to signature-based bridges while acknowledging explicit trade-offs.

Key Properties:

- **Cost reduction:** 95%+ savings on Solana storage via compressed accounts (~15,000 vs ~1,600,000 lamports per deposit record)
- **Verification:** GROTH16 proofs (~100-bit security on BN254) replace multisig attestation for withdrawal authorization
- **Latency:** ~20–25 seconds end-to-end (competitive with intent-based bridges)
- **Replay protection:** POSEIDON-based nullifiers with on-chain tracking

Explicit Limitations:

- **Throughput:** ~12–20 withdrawals per minute per direction (sequential IMT updates)
- **EVM costs:** Withdrawal verification costs ~\$4–6 at 30 gwei, dominating total transfer cost
- **Trust assumptions:** GROTH16 trusted setup, Light Protocol implementation, Photon indexer availability, relayer liveness

The protocol shifts the trust model from “honest majority of signers” to “cryptographic soundness plus infrastructure liveness.” A compromised relayer cannot forge proofs or double-spend, but can censor transactions or extract MEV through reordering.

Contents

1	Problem Statement	5
1.1	Prohibitive On-Chain Storage Costs	5
1.2	Trust in Intermediaries	5
1.3	Replay Attacks and Double-Spending	6
2	Why Solana $\leftrightarrow$ EVM Bridging is Hard	6
2.1	Consensus Incompatibility	6
2.2	Address Space and Field Constraints	7
2.3	Transaction Size Constraints	7
2.4	The Oracle Problem	7
3	Existing Bridge Models	8
3.1	Wormhole (Portal Bridge)	8
3.2	LayerZero	8
3.3	deBridge	8
3.4	Allbridge Core	9
3.5	Meridian Link Positioning	9
4	Architecture Proposal	9
4.1	System Architecture Overview	10
4.2	Component 1: Solana Bridge Program	10
4.2.1	Account Structures	10
4.2.2	Light Protocol Integration	11
4.3	Component 2: EVM Bridge Contract	11
4.3.1	Contract State	11
4.3.2	Core Functions	11
4.4	Component 3: Zero-Knowledge Circuits	12
4.4.1	Solana Deposit Proof Circuit	12
4.4.2	Ethereum Deposit Proof Circuit	12
4.5	Component 4: Relayer Service	13
4.5.1	Architecture Requirements	13
4.5.2	Solana $\rightarrow$ EVM Flow	13
4.5.3	EVM $\rightarrow$ Solana Flow	13
5	Trust and Security Model	14
5.1	Threat Model	14
5.1.1	Malicious Relayer	14
5.1.2	Trusted Setup Compromise	14
5.2	Trust Assumptions	14
5.3	Security Properties	15
5.3.1	Safety	15
5.3.2	Liveness	15
5.3.3	Replay Protection	15
5.4	Groth16 Security Analysis	16
6	Message Flow	16
6.1	Solana $\rightarrow$ EVM Transfer	17
6.2	EVM $\rightarrow$ Solana Transfer	18
7	Failure Cases and Recovery	19
7.1	Network Failures	19

7.1.1	RPC Unavailability	19
7.1.2	Block Reorganization	19
7.2	Relayer Failures	19
7.2.1	Relayer Downtime	19
7.2.2	Relayer Compromise	19
7.3	Recovery Matrix	20
8	Gas, Latency, and Finality	20
8.1	Cost Analysis	20
8.1.1	Solana Costs	20
8.1.2	EVM Costs	20
8.1.3	Fee Model	20
8.2	Latency Analysis	21
8.2.1	Solana Commitment Levels	21
8.2.2	End-to-End Latency	21
8.3	Throughput Considerations	21
8.3.1	Sequential Processing Limitation	21
8.3.2	Future Optimization: Concurrent Merkle Trees	22
9	Comparison with Existing Solutions	22
9.1	Feature Comparison Matrix	22
9.2	Security Model Analysis	22
9.3	Cost and Latency Summary	23
10	Future Work	23
10.1	Near-Term (0–6 months)	23
10.1.1	Mainnet Deployment	23
10.1.2	Fee Structure and Rate Limiting	23
10.1.3	Additional Chain Support	23
10.2	Medium-Term (6–18 months)	23
10.2.1	Concurrent Merkle Trees	23
10.2.2	Hardware-Accelerated Proving	24
10.2.3	Decentralized Relayer Network	24
11	Conclusion	24
11.1	Contributions	24
11.2	Limitations and Trade-offs	25
11.3	Open Questions	25
11.4	Invitation	25
	References	26
A	Cryptographic Primitives	27
A.1	Poseidon Hash Function	27
A.2	Groth16 Proof System	27
A.3	BN254 Curve Parameters	27
B	Circuit Specifications	27
B.1	Solana Deposit Proof Circuit	27
B.2	Ethereum Deposit Proof Circuit	28

1 Problem Statement

Cross-chain bridges face three fundamental challenges that current solutions fail to adequately address.

1.1 Prohibitive On-Chain Storage Costs

Most bridges store transaction records directly on-chain, incurring substantial storage costs that scale linearly with usage. On Solana, creating a standard Program Derived Account (PDA) requires approximately 1,600,000 lamports (~\$0.25 at current prices) for rent-exemption of a 100-byte account [1]. For a bridge processing thousands of transactions daily, these costs become economically prohibitive.

Meridian Link’s approach: By leveraging Light Protocol’s state compression, we reduce storage costs by over 95%. A compressed account costs approximately 15,000 lamports versus 1,600,000 lamports for a traditional PDA, a 99.06% reduction [2]. This is achieved by storing only a 32-byte hash of the account data in a sparse Merkle tree, with the full account data recorded on the Solana ledger (not in expensive account space).

Table 1: Storage Cost Comparison

Account Type	Traditional Cost	Compressed Cost	Savings
Token Account	~2,000,000 lamports	~5,000 lamports	99.75%
100-byte PDA	~1,600,000 lamports	~15,000 lamports	99.06%
Nullifier PDA	~890,880 lamports	~15,000 lamports	98.32%

1.2 Trust in Intermediaries

Existing bridges require users to trust various intermediaries:

- **Multisig committees:** Wormhole’s Guardian Network requires 13-of-19 guardians to attest to cross-chain messages [3]. A compromise of 7 guardians enables arbitrary message forgery.
- **Oracle networks:** LayerZero’s Decentralized Verifier Networks (DVNs) introduce oracle trust assumptions [4].
- **Liquidity providers:** Pool-based bridges like Allbridge require trust in pool operators and expose users to impermanent loss.

Meridian Link’s approach: We employ GROTH16 zk-SNARK proofs [5] for verification, reducing trust to cryptographic assumptions. The withdrawal of tokens on the destination chain requires a valid zero-knowledge proof demonstrating that:

1. A corresponding deposit exists on the source chain
2. The deposit is included in the protocol’s state tree
3. The nullifier has not been previously used (preventing double-spending)

The relay cannot forge proofs, computational infeasibility under the discrete logarithm assumption on BN254 (~100-bit security). However, trust assumptions remain:

- **Trusted setup:** GROTH16 requires a circuit-specific ceremony; a compromised setup enables proof forgery

- **Liveness:** The relayer must be operational; if it fails, users reclaim deposits via time-lock mechanism
- **Infrastructure:** Light Protocol and Photon indexer must function correctly

1.3 Replay Attacks and Double-Spending

Without proper replay protection, a single deposit could be used to withdraw tokens multiple times. Many bridges rely on centralized tracking systems that introduce single points of failure.

Meridian Link’s approach: Each bridge request generates a unique nullifier computed via POSEIDON hashing [6]:

$$\text{nullifier} = \text{POSEIDON}_7(\text{depId}, \text{srcChain}, \text{destChain}, \text{amt}, \text{mint}, \text{owner}, \text{ts}) \quad (1)$$

This nullifier is:

- Computed deterministically from the deposit parameters (cannot be guessed or forged)
- Stored on-chain after each withdrawal (preventing reuse)
- Verified as part of the zero-knowledge proof (cryptographically bound to the deposit)

The POSEIDON hash function is specifically chosen for its efficiency in arithmetic circuits, requiring only ~ 300 constraints per hash operation compared to $\sim 25,000$ for SHA-256 [7].

2 Why Solana $\leftrightarrow$ EVM Bridging is Hard

Cross-chain communication between Solana and EVM chains presents unique technical challenges that distinguish it from bridging between homogeneous blockchain families.

2.1 Consensus Incompatibility

Solana employs a hybrid consensus mechanism combining Proof of History (PoH) with Tower BFT [8]:

- **Slot time:** 400ms
- **Processed commitment:** Transaction received by leader (~ 400 ms)
- **Confirmed commitment:** Supermajority (66%+) vote confirmation (~ 2 –3 seconds)
- **Finalized commitment:** 31+ confirmed blocks built atop (~ 15 –30 seconds)

Ethereum (post-Merge) uses Gasper consensus [9]:

- **Block time:** 12 seconds
- **Epoch:** 32 slots (6.4 minutes)
- **Finality:** 2 epochs (~ 12.8 minutes under normal conditions)

There is no native mechanism for one chain to verify the consensus of the other. Light client implementations would require:

- Solana to verify Ethereum’s BLS aggregate signatures over the validator set
- Ethereum to verify Solana’s Tower BFT votes and PoH chain

Neither chain natively supports the cryptographic primitives required for the other’s consensus verification, making true trustless light clients impractical without significant protocol changes.

2.2 Address Space and Field Constraints

Circom’s field limitation: Zero-knowledge circuits in Circom operate over the BN254 (alt_bn128) elliptic curve scalar field, which has a prime order of approximately 2^{254} [10]. This creates challenges when handling:

- **Ethereum addresses:** 160 bits (20 bytes) , fits within the field
- **Solana addresses:** 256 bits (32 bytes) , exceeds the field size

Meridian Link’s solution: For the Ethereum Deposit Proof circuit, we split 256-bit values into two 128-bit chunks:

```
signal input depositorHigh; // Upper 128 bits
signal input depositorLow;  // Lower 128 bits
```

Listing 1: Address Splitting in Circom

2.3 Transaction Size Constraints

Solana enforces a strict 1,232-byte limit on transaction size [11]. A typical cross-chain withdrawal transaction must include:

- GROTH16 proof: 256 bytes (compressed)
- Public inputs: ~ 128 bytes
- Merkle proof path: Variable (up to 832 bytes for 26 levels)
- Account metadata: ~ 100 bytes

Light Protocol’s validity proof optimization reduces the Merkle proof requirement to a constant 128 bytes by leveraging the on-chain state root, making cross-chain transactions feasible within Solana’s constraints [2].

2.4 The Oracle Problem

Most existing Solana-EVM bridges rely on oracle-based attestation:

- **Wormhole:** 19 Guardian nodes sign observed messages; 13/19 threshold for validity [3]
- **LayerZero:** Decentralized Verifier Networks (DVNs) attest to cross-chain messages [4]

These approaches inherit the security assumptions of the oracle network. If the oracle set is compromised (via key theft, coercion, or collusion), arbitrary messages can be forged.

Meridian Link’s approach: Rather than trusting oracles to attest to state, we generate cryptographic proofs that the state exists. The security reduces to:

1. The soundness of the GROTH16 proof system (~ 100 -bit security) [5]
2. The integrity of the on-chain state root (verified by Solana/Ethereum consensus)
3. The correctness of the circuit (auditable, deterministic)

3 Existing Bridge Models

3.1 Wormhole (Portal Bridge)

Architecture: Lock-and-mint model with Guardian Network attestation [3]

- **TVL:** ~\$2 billion (as of 2024)
- **Trust model:** 19 Guardian nodes operated by institutional validators; 13/19 threshold
- **Mechanism:**
 1. User locks tokens on source chain
 2. Guardians observe and sign VAA (Verified Action Approval)
 3. User submits VAA to destination chain
 4. Destination chain mints wrapped tokens

Security incident: In February 2022, an attacker exploited a signature verification vulnerability, stealing \$326 million in wrapped ETH [12].

Limitations:

- Wrapped tokens introduce additional smart contract risk
- Guardian compromise enables arbitrary minting
- Latency of 15–20 minutes for finality

3.2 LayerZero

Architecture: Ultra Light Node (ULN) with configurable security [4]

- **TVL:** N/A (messaging protocol, not custodial)
- **Trust model:** Oracle + Relay separation; Decentralized Verifier Networks (DVNs)
- **Mechanism:**
 1. Source chain emits message
 2. DVNs verify and sign the message
 3. Relay submits message to destination
 4. Destination chain verifies DVN signatures

Limitations:

- Security depends on DVN selection and configuration
- Application developers must choose appropriate security settings
- No native support for Solana (requires custom adapters)

3.3 deBridge

Architecture: Intent-based Deferred Liquidity Network (DLN) [13]

- **TVL:** Growing (no pooled liquidity model)
- **Trust model:** Authenticated messages via validator signatures

- **Mechanism:**

1. User creates intent (desired outcome)
2. Solvers compete to fulfill the intent
3. Settlement occurs via authenticated messages

Inspiration for Meridian Link: deBridge’s no-wrapped-token approach and real-time execution model influenced our design. However, deBridge still relies on validator signatures rather than cryptographic proofs.

3.4 Allbridge Core

Architecture: Liquidity pool model with native stablecoin transfers [14]

- **TVL:** ~\$25 million
- **Trust model:** Pool operators and liquidity providers
- **Mechanism:**
 1. User deposits tokens into source pool
 2. Equivalent tokens released from destination pool
 3. Pool rebalancing via cross-chain messages

Limitations:

- Liquidity fragmentation across pools
- Exposure to impermanent loss for LPs
- Pool depletion risk during high-volume periods

3.5 Meridian Link Positioning

MERIDIAN LINK differentiates from the above models through ZK proof-based verification rather than signature aggregation, combined with compressed on-chain storage via Light Protocol. A detailed quantitative comparison is provided in Section 9.

4 Architecture Proposal

MERIDIAN LINK consists of five primary components working in concert to enable cross-chain transfers with reduced trust assumptions.

4.1 System Architecture Overview

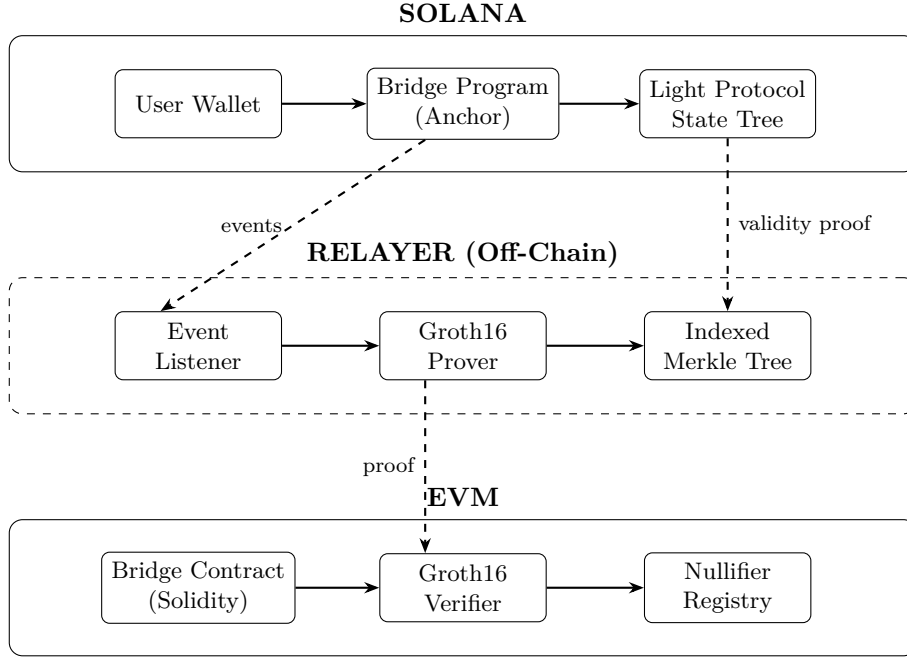


Figure 1: Meridian Link System Architecture

4.2 Component 1: Solana Bridge Program

The Solana bridge program is implemented using the Anchor framework and integrates with Light Protocol for state compression.

4.2.1 Account Structures

The program maintains four primary account types:

Table 2: Solana Bridge Account Types

Account	Purpose	Storage
BridgeState	Global protocol state	Traditional PDA
TokenBridge	Cross-chain token mapping	Traditional PDA
DepositRecord	Per-deposit details	Compressed account
WithdrawalProof	Groth16 proof data	Traditional PDA

The **DepositRecord** is stored as a Light Protocol compressed account, containing:

- **owner:** Depositor's Solana public key (32 bytes)
- **source_chain_id, dest_chain_id:** Chain identifiers (4 bytes each)
- **dest_chain_addr:** Recipient address on target chain (string)
- **mint:** Token mint address (32 bytes)
- **amount:** Transfer amount in smallest units (8 bytes)
- **timestamp:** Unix timestamp (8 bytes)

- **deposit_id:** Unique monotonic identifier (8 bytes)

4.2.2 Light Protocol Integration

Light Protocol enables state compression through the following mechanism [2]:

1. **Account Creation:** Instead of creating a full Solana account (1.6M lamports), we create a compressed account (~15K lamports)
2. **Hash Storage:** Only the 32-byte hash of the account data is stored in a sparse Merkle tree
3. **Data Availability:** The full account data is recorded on the Solana ledger (calldata)
4. **Validity Proofs:** Any read/write requires a 128-byte validity proof against the on-chain root

Tree specifications:

- **Depth:** 26 levels
- **Capacity:** ~67 million compressed accounts per tree
- **Proof size:** 128 bytes (constant, regardless of tree depth)
- **Hash function:** POSEIDON (circuit-friendly)

4.3 Component 2: EVM Bridge Contract

The EVM bridge contract is implemented in Solidity 0.8.20 with OpenZeppelin security primitives (SafeERC20, Ownable).

4.3.1 Contract State

Table 3: EVM Contract State Variables

Variable	Type	Purpose
<code>verifier</code>	<code>SolDepositVerifier</code>	Groth16 proof verification
<code>usedNullifiers</code>	<code>mapping(uint256 → bool)</code>	Replay protection registry
<code>tokenMapping</code>	<code>mapping(string → address)</code>	Solana mint → ERC20
<code>authorizedRelayers</code>	<code>mapping(address → bool)</code>	Access control

4.3.2 Core Functions

Deposit (EVM → Solana):

1. Validate chain IDs (source = current chain, dest = Solana)
2. Transfer ERC20 tokens from user to contract via `safeTransferFrom`
3. Emit `EthDeposit` event with deposit details and monotonic `depositId`

Process Withdrawal (Solana → EVM):

1. Verify `sourceChainId == SOLANA_CHAIN_ID`
2. Check nullifier not in `usedNullifiers` (replay protection)
3. Mark nullifier as used **before** external calls (CEI pattern)

4. Verify GROTH16 proof via `verifier.verifyProof( $\pi_A$ ,  $\pi_B$ ,  $\pi_C$ , publicSignals)`
5. Transfer tokens to recipient via `safeTransfer`
6. Emit `WithdrawalProcessed` event

The CEI (Checks-Effects-Interactions) pattern prevents reentrancy attacks by updating state before external calls.

4.4 Component 3: Zero-Knowledge Circuits

The protocol employs two Circom circuits for bidirectional verification.

4.4.1 Solana Deposit Proof Circuit

Purpose: Prove that a deposit exists in Light Protocol’s state tree without revealing private deposit details.

Table 4: Solana Deposit Proof Circuit Parameters

Parameter	Value
Merkle tree depth	26 levels
Hash function	POSEIDON (2-ary for tree, 7-ary for nullifier)
Constraint count	~15,000 R1CS
Proof size	256 bytes (Groth16)

Circuit operations:

1. **Merkle inclusion:** Verify `accountHash` exists in tree with root `stateRoot` using 26-level proof
2. **Nullifier computation:**

$$\text{nullifier} = \text{POSEIDON}_7(\text{depositId}, \text{srcChain}, \text{destChain}, \text{amount}, \text{mint}, \text{owner}, \text{ts})$$

3. **Commitment binding:**

$$\text{commitment} = \text{POSEIDON}_5(\text{amount}, \text{destAddr}, \text{destChain}, \text{depositId}, \text{nullifier})$$

4. **Constraint validation:** Assert `sourceChainId == 1` (Solana) and `amount > 0`

4.4.2 Ethereum Deposit Proof Circuit

Purpose: Prove deposit inclusion in an Indexed Merkle Tree (IMT) and compute the new root after nullifier insertion.

Design challenge: Circom operates over the BN254 scalar field ($\sim 2^{254}$), but some values exceed this bound. Solution: split 256-bit values into two 128-bit field elements.

Table 5: Ethereum Deposit Proof Circuit Parameters

Parameter	Value
IMT depth	32 levels
Constraint count	~20,000 R1CS
Proof size	256 bytes (Groth16)

Indexed Merkle Tree (IMT): Unlike standard Merkle trees, IMTs maintain a sorted linked-list structure within the tree, enabling efficient non-membership proofs [15]. This allows proving a nullifier has NOT been used, required for preventing replay on the EVM→Solana path.

4.5 Component 4: Relay Service

The relay is a persistent off-chain service that bridges the gap between chains.

4.5.1 Architecture Requirements

- **Deployment:** Requires persistent infrastructure (not serverless)
- **Listeners:** WebSocket connections to both Solana and EVM RPCs
- **State:** Maintains local Indexed Merkle Tree for EVM deposits
- **Proving:** Integrates snarkjs for GROTH16 proof generation

4.5.2 Solana → EVM Flow

The relay processes Solana deposits through the following steps:

1. **Event detection:** Listen for `DepositEvent` emissions from the Solana bridge program
2. **Account retrieval:** Fetch compressed account data from Light Protocol RPC
3. **Data decoding:** Deserialize Borsh-encoded deposit record
4. **Proof acquisition:** Request validity proof from Photon indexer (128-byte proof against state root)
5. **Circuit input preparation:** Convert addresses to field elements, compute path indices
6. **Proof generation:** Execute snarkjs GROTH16 prover (~2–5 seconds)
7. **Submission:** Call `processWithdrawal` on EVM bridge contract
8. **Confirmation:** Wait for EVM transaction confirmation

Latency breakdown: ~5s (Solana finality) + ~1s (event detection) + ~3s (proof generation) + ~12s (EVM confirmation) = ~20–25s total

4.5.3 EVM → Solana Flow

The reverse flow introduces additional complexity due to IMT maintenance:

1. **Event detection:** Listen for `EthDeposit` events from EVM bridge contract
2. **Nullifier computation:** Hash deposit parameters via POSEIDON₉
3. **Non-membership verification:** Query local IMT to confirm nullifier not present
4. **Proof generation:** Generate IMT insertion proof via snarkjs (~3–5 seconds)
5. **Submission:** Call `withdraw` on Solana bridge program with proof and new root
6. **IMT update:** Insert nullifier into local IMT after successful withdrawal

Critical constraint: IMT updates are sequential, each proof depends on the previous root. This limits throughput to ~12–20 withdrawals per minute per direction.

5 Trust and Security Model

5.1 Threat Model

We consider the following adversarial scenarios and their mitigations.

5.1.1 Malicious Relayer

Threat: The relayer attempts to steal funds, double-process deposits, or extract value.

What the relayer CANNOT do (safety guarantees):

- **Cannot forge proofs:** GROTH16 soundness ensures only valid proofs verify (~ 100 -bit security on BN254)
- **Cannot double-spend:** Nullifiers are stored on-chain; the contract rejects reused nullifiers
- **Cannot modify amounts:** Public inputs (amount, recipient) are verified by the circuit
- **Cannot create funds:** Withdrawals require valid deposit proofs

What the relayer CAN do (attack surface):

- **Censorship:** Refuse to process specific users' deposits indefinitely
- **Reordering:** Process transactions in self-beneficial order during congestion
- **MEV extraction:** Front-run large transfers by observing pending deposits before processing
- **Delay attacks:** Withhold proofs while trading on information asymmetry
- **Selective service:** Prioritize high-fee transactions over fairness

Classification: Safety is preserved; liveness and fairness are NOT guaranteed under relayer compromise. Users can recover funds via time-lock mechanism if relayer fails completely.

5.1.2 Trusted Setup Compromise

Threat: GROTH16 trusted setup ceremony was compromised, allowing proof forgery.

Mitigation:

- **Powers of tau:** Phase 1 uses community-contributed randomness from Hermez/Zcash ceremonies
- **Circuit-specific phase 2:** Multi-party computation with public transcript
- **Verification:** Setup artifacts are deterministically verifiable

Residual risk: If ALL ceremony participants colluded or were compromised, arbitrary proofs can be forged. This is a fundamental GROTH16 limitation.

5.2 Trust Assumptions

MERIDIAN LINK operates under the following explicit trust assumptions:

Table 6: Trust Assumptions Matrix

Component	Trust Assumption	Failure Mode	Severity
Solana Consensus	Honest supermajority	State reversion	Critical
EVM Consensus	Honest supermajority	State reversion	Critical
GROTH16 Soundness	Discrete log on BN254	Forged proofs	Critical
Trusted Setup	≥ 1 honest participant	Proof forgery	Critical
POSEIDON Security	Collision resistance	Double-spend	High
Light Protocol	Correct implementation	Invalid proofs	High
Photon Indexer	Data availability	Liveness failure	Medium
Relayer	Liveness + fairness	Censorship, MEV	Medium

Note on BN254 security: Recent cryptanalysis (Kim-Barbulescu, 2016) demonstrates sub-exponential attacks on BN-family curves [17]. Conservative estimates place BN254 security at ~ 100 – 110 bits, not 128 bits as sometimes claimed.

5.3 Security Properties

5.3.1 Safety

Definition: No funds can be stolen or created from nothing.

Guarantee: A withdrawal on the destination chain requires:

1. A valid GROTH16 proof (unforgeable without the witness)
2. A state root matching the on-chain root (consensus-protected)
3. An unused nullifier (on-chain tracking)

The security reduces to the soundness of GROTH16 (~ 100 -bit on BN254) and the integrity of chain consensus.

5.3.2 Liveness

Definition: Valid deposits will eventually be processed.

Guarantee: Under normal operation, the relayer processes deposits within ~ 20 – 25 seconds. If the relayer fails:

1. User funds remain locked in source chain vault
2. After time-lock expiry, users can reclaim funds directly
3. No permanent loss of funds

5.3.3 Replay Protection

Definition: Each deposit can only be withdrawn once.

Guarantee: The nullifier $H(\text{depositId}, \text{sourceChainId}, \text{destChainId}, \text{amount}, \text{mint}, \text{owner}, \text{timestamp})$ is:

- Deterministic (same deposit $\rightarrow$ same nullifier)
- Unique (collision resistance of POSEIDON)
- Tracked on-chain (cannot be reused)

5.4 Groth16 Security Analysis

GROTH16 [5] provides the following security properties on the BN254 curve:

Soundness: An adversary cannot produce a valid proof for a false statement except with negligible probability. On BN254, this is approximately 2^{-100} to 2^{-110} based on current best-known attacks (not 2^{-128} as sometimes claimed).

Knowledge soundness: An adversary producing a valid proof must “know” a valid witness (extractability).

Zero-knowledge: The proof reveals nothing about the witness beyond the statement’s validity.

Proof size: Constant 256 bytes (2 G_1 points + 1 G_2 point), regardless of circuit complexity.

Verification cost: $\sim 200,000$ gas on EVM (6 pairing operations on BN254 precompiles).

Trusted setup requirements:

1. **Phase 1 (Powers of Tau):** Universal, reusable across circuits. We leverage Hermez/Z-cash community ceremonies with thousands of participants.
2. **Phase 2 (Circuit-specific):** Requires fresh randomness per circuit. A single honest participant suffices for security.

Why we chose Groth16 despite trusted setup limitation:

- Smallest proof size (256 bytes vs. $\sim 50\text{KB}$ for STARKs)
- Fastest verification ($\sim 200\text{K}$ gas vs. $\sim 1\text{M}+$ gas for STARKs)
- Mature tooling (circom, snarkjs)
- EVM precompile support (EIP-196, EIP-197)

6 Message Flow

6.1 Solana → EVM Transfer

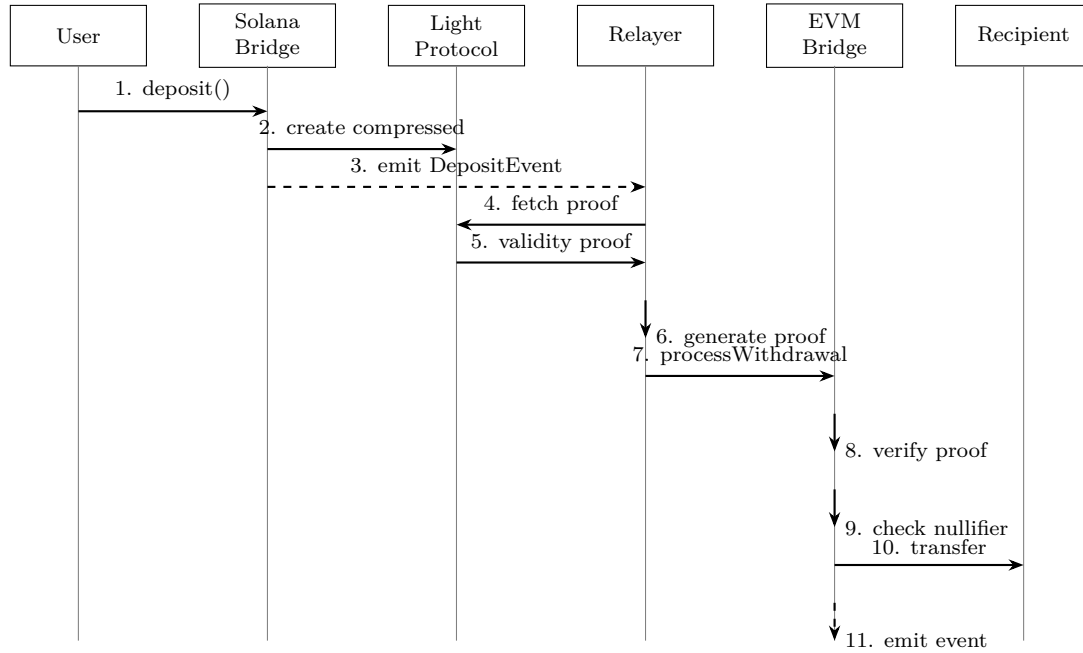


Figure 2: Solana → EVM Message Flow

Step-by-step breakdown:

1. **User initiates deposit:** Calls `deposit(amount, destChainAddr, linkHash)` on Solana bridge
2. **Compressed account created:** Light Protocol creates a compressed account with deposit details (~15,000 lamports)
3. **Event emitted:** The Solana program emits a `DepositEvent` with the compressed account address
4. **Relayer fetches proof:** Queries Photon indexer for the validity proof of the compressed account
5. **Proof received:** 128-byte validity proof against current state root
6. **Groth16 proof generated:** Relayer runs `snarkjs` with circuit inputs (~2–5 seconds)
7. **Withdrawal submitted:** Relayer calls `processWithdrawal(record, proof)` on EVM bridge
8. **Proof verified:** EVM contract verifies GROTH16 proof (~200,000 gas)
9. **Nullifier checked:** Contract confirms nullifier hasn't been used
10. **Tokens transferred:** ERC20 tokens sent to recipient address
11. **Event emitted:** `WithdrawalProcessed` event for tracking

Total latency: ~20–25 seconds (5s Solana finality + 2–5s proof gen + 12s EVM confirmation)

6.2 EVM → Solana Transfer

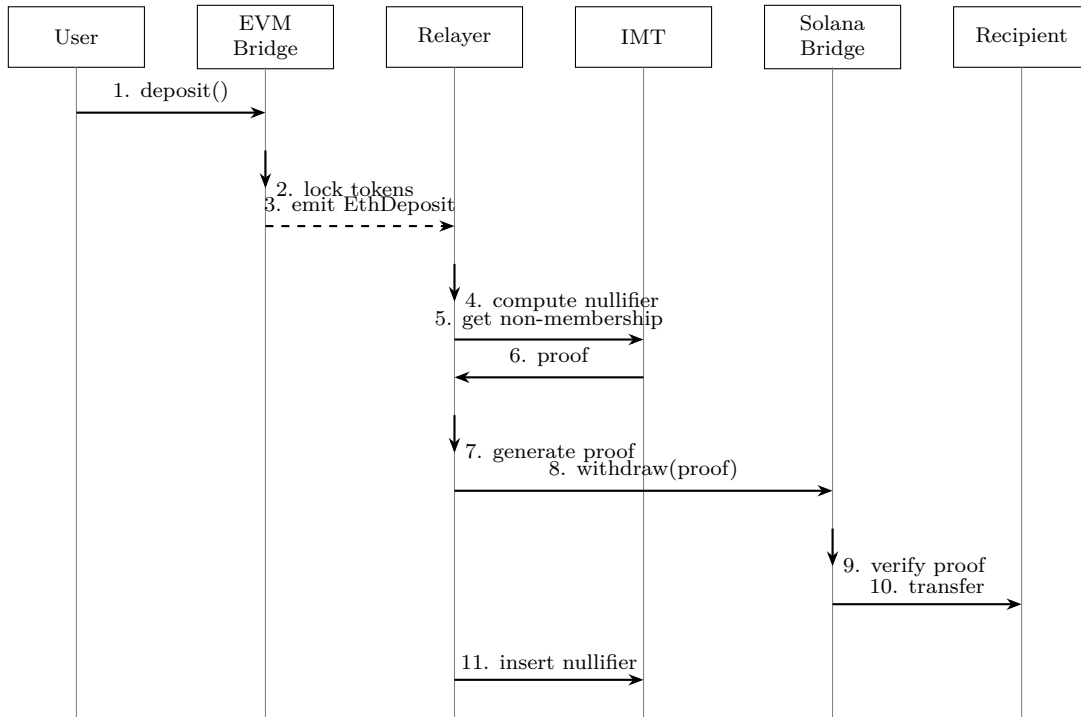


Figure 3: EVM → Solana Message Flow

Step-by-step breakdown:

1. **User initiates deposit:** Calls `deposit()` on EVM bridge contract
2. **Tokens locked:** ERC20 tokens transferred to bridge contract
3. **Event emitted:** Bridge emits `EthDeposit` event with deposit details
4. **Nullifier computed:** Relay hashes deposit parameters via `POSEIDON9`
5. **Non-membership query:** Relay queries local IMT to confirm nullifier not present
6. **Proof returned:** IMT returns non-membership proof (predecessor/successor linkage)
7. **Groth16 proof generated:** Relay generates IMT insertion proof (~3–5 seconds)
8. **Withdrawal submitted:** Relay calls `withdraw(proof, newRoot, nullifier)` on Solana
9. **Proof verified:** Solana program verifies GROTH16 proof
10. **Tokens transferred:** SPL tokens sent to recipient address
11. **IMT updated:** Nullifier inserted into local IMT after successful withdrawal

Total latency: ~22–25 seconds (12s EVM confirmation + 3–5s proof gen + 5s Solana finality)

Key differences from Solana→EVM:

- **Indexed Merkle Tree (IMT):** Used for non-membership proofs (proving nullifier hasn't been used)
- **IMT maintained off-chain:** Relay stores IMT locally; root hash tracked on Solana

- **Insertion after withdrawal:** Nullifier inserted into IMT only after successful Solana withdrawal

7 Failure Cases and Recovery

7.1 Network Failures

7.1.1 RPC Unavailability

Scenario: Solana or EVM RPC endpoints become unreachable.

Impact: Relay cannot fetch events or submit transactions.

Recovery:

- Relay implements exponential backoff retry logic
- Multiple RPC endpoints configured as fallbacks
- User funds remain safe in source chain contracts

7.1.2 Block Reorganization

Scenario: A confirmed block is reverted due to chain reorganization.

Impact: Deposit events may be “undone” if processed before finality.

Mitigation:

- Relay waits for finalized commitment on Solana (~15–30 seconds)
- Relay waits for sufficient confirmations on EVM (12+ blocks)
- Deposits processed only after achieving finality threshold

7.2 Relay Failures

7.2.1 Relay Downtime

Scenario: Relay service crashes or becomes unresponsive.

Impact: Deposits are not processed; funds remain locked.

Recovery:

1. **Time-lock mechanism:** Deposits have a maximum processing window (e.g., 24 hours)
2. **User reclaim:** After time-lock expiry, users can call `reclaimDeposit()` on source chain
3. **No fund loss:** Funds are never permanently stuck

7.2.2 Relay Compromise

Scenario: Attacker gains control of relay private keys.

Impact: Limited to denial of service.

Why funds are safe:

- Cannot forge GROTH16 proofs (requires valid witness)
- Cannot modify withdrawal amounts (verified by circuit)

- Cannot double-spend (nullifiers tracked on-chain)
- Can only refuse to process (liveness attack, not safety)

7.3 Recovery Matrix

Table 7: Failure Recovery Matrix

Failure Mode	Impact	User Action	Recovery Time
RPC down	Delayed processing	None	Minutes
Block reorg	Potential retry	None	Minutes
Relayer down	Stuck deposits	Reclaim after timeout	Hours
Relayer compromise	DoS only	Wait for recovery	Hours
Tree full	New deposits fail	None (auto-rollover)	Minutes
Queue full	New deposits fail	None (forester catches up)	Hours

8 Gas, Latency, and Finality

8.1 Cost Analysis

8.1.1 Solana Costs

Table 8: Solana Operation Costs

Operation	Traditional	Compressed	Savings
Create deposit record	~1,600,000 lamports	~15,000 lamports	99.06%
Create token account	~2,000,000 lamports	~5,000 lamports	99.75%
Store nullifier	~890,880 lamports	~15,000 lamports	98.32%
Transaction fee	~5,000 lamports	~5,000 lamports	0%

Total deposit cost (Solana side): ~20,000 lamports (~\$0.003 at SOL=\$150)

8.1.2 EVM Costs

Table 9: EVM Operation Costs

Operation	Gas Used	Cost @ 30 gwei
GROTH16 verification	~200,000	~\$3–5
Nullifier storage	~20,000	~\$0.30
ERC20 transfer	~65,000	~\$1
Event emission	~5,000	~\$0.08
Total withdrawal	~290,000	~\$4–6

Note: Costs vary significantly based on network congestion and gas prices.

8.1.3 Fee Model

The relayer deducts fees from the transfer amount:

$$\text{net_amount} = \text{gross_amount} - \text{gas_fee} - \text{relayer_fee} - \text{LP_fee} \quad (2)$$

Where:

- **gas_fee:** Estimated destination chain transaction cost
- **relayer_fee:** Flat fee for relayer infrastructure ($\sim \$0.50$)
- **LP_fee:** Liquidity provider incentive (0.1% of amount)

Example: 1000 USDC transfer, Solana→Ethereum

- Gas fee: $\sim \$5.00$
- Relayer fee: $\$0.50$
- LP fee: $\$1.00$
- **Net received:** 993.50 USDC

8.2 Latency Analysis

8.2.1 Solana Commitment Levels

Table 10: Solana Commitment Levels

Level	Time	Security
Processed	$\sim 400\text{ms}$	Single validator confirmation
Confirmed	$\sim 2\text{--}3\text{s}$	Supermajority (66%+) vote
Finalized	$\sim 15\text{--}30\text{s}$	31+ confirmed blocks

MERIDIAN LINK waits for **finalized** commitment to ensure deposit cannot be reverted.

8.2.2 End-to-End Latency

Solana $\rightarrow$ EVM:

Stage	Duration
Solana deposit confirmation	$\sim 5\text{s}$ (finalized)
Event detection	$\sim 1\text{s}$
Proof generation (snarkjs)	$\sim 2\text{--}5\text{s}$
EVM transaction broadcast	$\sim 1\text{s}$
EVM confirmation	$\sim 12\text{s}$ (1 block)
Total	$\sim 20\text{--}25\text{s}$

8.3 Throughput Considerations

8.3.1 Sequential Processing Limitation

Current implementation uses **Indexed Merkle Trees** which require sequential updates:

- Each IMT insertion depends on the previous root
- Parallel withdrawals would create conflicting root updates

- Theoretical maximum: ~ 1 withdrawal per proof generation cycle ($\sim 3\text{--}5\text{s}$)

Practical throughput: $\sim 12\text{--}20$ withdrawals per minute per direction

8.3.2 Future Optimization: Concurrent Merkle Trees

Light Protocol supports **Concurrent Merkle Trees** [16] which allow parallel updates:

- Multiple proofs can reference the same root
- Conflicts resolved via changelog mechanism
- Theoretical improvement: $10\text{--}100\times$ throughput

This optimization is planned for future releases.

9 Comparison with Existing Solutions

9.1 Feature Comparison Matrix

Table 11: Feature Comparison with Existing Bridges

Feature	Meridian Link	Wormhole	LayerZero	deBridge	Allbridge
Trust Model	ZK Proofs	13/19 Guardians	Oracle + DVN	Validators	Pool Ops
Verification	Groth16	ECDSA Sigs	ECDSA Sigs	ECDSA Sigs	Centralized
Wrapped Tokens	No	Yes	Configurable	No	No
Storage	Compressed	Full accounts	Minimal	Minimal	Pool state
Latency	$\sim 20\text{--}25\text{s}$	15–20 min	Variable	Real-time	Fast
User Recovery	Time-lock	Guardian	DVN-dependent	Support	Support
Solana Support	Native	Native	Via adapters	Native	Native
TVL	N/A (new)	$\sim \$2\text{B}$	N/A	Growing	$\sim \$25\text{M}$

9.2 Security Model Analysis

Each bridge makes different trust assumptions. We characterize them by their **security bottleneck**:

Table 12: Security Bottleneck Analysis

Bridge	Security Bottleneck	Attack Threshold
Wormhole	Guardian honesty	7/19 compromised guardians
LayerZero	DVN + Oracle honesty	Application-dependent
deBridge	Validator honesty	Validator set compromise
Allbridge	Pool operator honesty	Single operator compromise
MERIDIAN LINK	Groth16 + trusted setup	$\sim 2^{-100}$ forgery probability

Honest assessment of Meridian Link:

Strengths over signature-based bridges:

- Relay cannot forge proofs (cryptographic, not economic security)
- User recovery via time-lock (no intermediary required)
- No wrapped tokens (native asset transfers)

Weaknesses compared to established bridges:

- Trusted setup introduces systemic risk if ceremony compromised
- Lower throughput ($\sim 12\text{--}20/\text{min}$ vs. unlimited for signature verification)
- Less battle-tested (new protocol)
- Higher EVM costs (\$4–6) than some competitors (\$2–5)

9.3 Cost and Latency Summary

Table 13: Cost and Latency Comparison

Bridge	Total Cost (1000 USDC)	Latency	Throughput
Wormhole	$\sim \$5\text{--}11$	15–20 min	High
LayerZero	$\sim \$3\text{--}9$	Variable	High
deBridge	$\sim \$2\text{--}6$	30s–2 min	High
Allbridge	$\sim \$5\text{--}15$	1–5 min	Medium
Meridian Link	$\sim \\$4\text{--}7$	20–25s	Low ($\sim 12\text{--}20/\text{min}$)

Honest cost analysis: MERIDIAN LINK’s Solana storage savings ($\sim \$0.007/\text{deposit}$) are meaningful at scale but negligible for individual users. EVM verification costs dominate. For transfers under \$500, the $\sim \$5$ fee represents 1%+ of value.

10 Future Work

10.1 Near-Term (0–6 months)

10.1.1 Mainnet Deployment

The current implementation operates on Solana devnet and Ethereum Sepolia testnet. Mainnet deployment requires:

- **Security audit:** Comprehensive audit of smart contracts and circuits
- **Trusted setup ceremony:** Production GROTH16 keys via multi-party computation
- **Gradual rollout:** Start with low limits, increase based on monitoring
- **Bug bounty program:** Incentivize security researchers

10.1.2 Fee Structure and Rate Limiting

Implement on-chain fee configuration with base fees, percentage fees, and transfer limits to manage liquidity and incentivize relayer operation.

10.1.3 Additional Chain Support

Priority EVM chains: Arbitrum One, Base, HyperLiquid, Optimism, Polygon zkEVM

10.2 Medium-Term (6–18 months)

10.2.1 Concurrent Merkle Trees

Replace Indexed Merkle Trees with Concurrent Merkle Trees [16]:

- **Parallel processing:** Multiple withdrawals processed simultaneously
- **Throughput improvement:** 10–100x increase in transactions per second
- **Backward compatible:** Existing nullifiers remain valid

10.2.2 Hardware-Accelerated Proving

Integrate GPU/FPGA acceleration for proof generation:

- **Current:** ~2–5s proof generation (CPU, snarkjs)
- **Target:** ~100–500ms proof generation (GPU, rapidsnark)
- **Benefit:** Reduced latency, higher throughput

10.2.3 Decentralized Relay Network

Transition from single relay to permissionless network with staking requirements. Open research questions include:

- **Incentive design:** How to reward liveness without enabling MEV extraction
- **Slashing conditions:** What constitutes slashable behavior when proof forgery is cryptographically impossible
- **Coordination:** How multiple relayers avoid duplicate proof submissions

11 Conclusion

MERIDIAN LINK presents an alternative approach to cross-chain bridging that shifts trust assumptions from signature-based attestation to zero-knowledge proof verification, combined with state compression for cost efficiency.

11.1 Contributions

1. **Cost Reduction:** Light Protocol state compression reduces Solana storage costs by ~99% per deposit record (~15K vs ~1.6M lamports). However, EVM verification costs (~\$4–6) dominate total transfer cost for typical transactions.
2. **Verification Model:** GROTH16 proofs replace multisig/oracle attestation. Safety (no fund theft) holds under ~100-bit BN254 security and trusted setup integrity. Liveness depends on relay operation.
3. **Replay Protection:** POSEIDON-based nullifiers with on-chain tracking prevent double-spending without centralized databases.
4. **User Recovery:** Time-lock mechanism allows users to reclaim deposits if relay fails, eliminating permanent fund loss scenarios (assuming contract remains accessible).

11.2 Limitations and Trade-offs

Table 14: Limitations and Mitigations

Trade-off	Impact	Mitigation
Trusted setup	Proof forgery if compromised	Multi-party computation
Throughput	~12–20 withdrawals/min	Concurrent Merkle Trees
EVM costs	~\$4–6 per withdrawal	Batching, L2 deployment
Proof latency	~2–5s added per transfer	Hardware acceleration
Relayer centralization	Censorship, MEV possible	Decentralized network
New protocol	Limited battle-testing	Audits, gradual rollout

11.3 Open Questions

Several aspects warrant further research and discussion:

1. **Decentralized relayer incentives:** What economic mechanisms ensure relayer liveness without enabling MEV extraction?
2. **Cross-chain finality semantics:** How should the protocol handle deep reorganizations on either chain?
3. **Circuit upgradeability:** How can circuits be upgraded without breaking existing deposits in flight?
4. **Proof aggregation:** Can multiple withdrawal proofs be batched to reduce per-transaction EVM costs?

11.4 Invitation

This protocol is open-source. We welcome technical review, security analysis, and implementation contributions. The codebase is available at the repository linked below.

References

- [1] Solana Documentation. “Rent.” <https://docs.solana.com/developing/intro/rent>
- [2] Light Protocol. “ZK Compression Whitepaper.” <https://www.zkcompression.com/references/whitepaper>
- [3] Wormhole. “Wormhole Whitepaper.” <https://wormhole.com/whitepaper.pdf>
- [4] LayerZero. “LayerZero V2 Whitepaper.” https://layerzero.network/publications/LayerZero_Whitepaper_V2.0.0.pdf
- [5] Groth, Jens. “On the size of pairing-based non-interactive arguments.” *Advances in Cryptology–EUROCRYPT 2016*.
- [6] Grassi, Lorenzo, et al. “Poseidon: A new hash function for zero-knowledge proof systems.” *USENIX Security 2021*.
- [7] Bowe, Sean. “BLS12-381: New zk-SNARK Elliptic Curve Construction.” *Zcash Blog*, 2017.
- [8] Yakovenko, Anatoly. “Solana: A new architecture for a high performance blockchain.” *Solana Whitepaper*, 2017.
- [9] Buterin, Vitalik, et al. “Combining GHOST and Casper.” *arXiv:2003.03052*, 2020.
- [10] Ethereum Foundation. “EIP-196: Precompiled contracts for addition and scalar multiplication on the elliptic curve alt_bn128.” 2017.
- [11] Solana Documentation. “Transaction Size Limits.” <https://docs.solana.com/developing/programming-model/transactions>
- [12] Rekt News. “Wormhole - REKT.” <https://rekt.news/wormhole-rekt/>, February 2022.
- [13] deBridge. “deBridge Documentation.” <https://docs.debridge.finance/>
- [14] Allbridge. “Allbridge Core Documentation.” <https://docs.allbridge.io/>
- [15] Aztec Protocol. “Indexed Merkle Tree.” https://docs.aztec.network/aztec/concepts/storage/trees/indexed_merkle_tree
- [16] Solana Documentation. “Concurrent Merkle Trees.” <https://solana.com/developers/guides/javascript/compressed-nfts>
- [17] Kim, Taechan and Barbulescu, Razvan. “Extended Tower Number Field Sieve: A New Complexity for the Medium Prime Case.” *CRYPTO 2016*.

A Cryptographic Primitives

A.1 Poseidon Hash Function

POSEIDON [6] is an algebraic hash function optimized for zero-knowledge proof systems:

- **Field:** BN254 scalar field (prime $\sim 2^{254}$)
- **Width:** Configurable (we use 7-ary for nullifiers, 2-ary for Merkle trees)
- **Rounds:** Full rounds + partial rounds (security parameter dependent)
- **Constraints:** ~ 300 R1CS constraints per hash (vs $\sim 25,000$ for SHA-256)

A.2 Groth16 Proof System

GROTH16 [5] provides succinct non-interactive arguments of knowledge (SNARKs):

- **Proof size:** 256 bytes (constant)
- **Verification:** 6 pairing operations ($\sim 200K$ gas on EVM)
- **Security:** ~ 100 – 110 bit on BN254 (see Section 5 for discussion)
- **Trusted setup:** Circuit-specific (phase 1: powers of tau, phase 2: circuit specific)

A.3 BN254 Curve Parameters

The BN254 (alt_bn128) curve used in Circom:

$$p = 21888242871839275222246405745257275088696311157297823662689037894645226208583$$

$$r = 21888242871839275222246405745257275088548364400416034343698204186575808495617$$

EVM precompiles (EIP-196, EIP-197) provide efficient operations on this curve.

B Circuit Specifications

B.1 Solana Deposit Proof Circuit

Parameter	Value
Merkle tree depth	26
Public inputs	4 (stateRoot, amount, destChainId, destChainAddr)
Private inputs	13
Public outputs	3 (nullifier, commitment, isValid)
Constraints	$\sim 15,000$ R1CS
Proving time	~ 2 – 5 s (CPU)
Proof size	256 bytes

B.2 Ethereum Deposit Proof Circuit

Parameter	Value
IMT depth	32
Public inputs	10 (deposit event fields)
Private inputs	5 (IMT proof data)
Public outputs	2 (nullifier, newRoot)
Constraints	~20,000 R1CS
Proving time	~3–5s (CPU)
Proof size	256 bytes

Document Version: 1.0

Last Updated: January 2026

License: MIT

Repository: <https://github.com/RippnerLabs/meridian-link>